

PROJETO SOBRA ZERO

Documento de Arquitetura e Requisitos — AEP 1º Bimestre

Curso: Engenharia de Software | Turma: ESOF4SNB | Unicesumar — Maringá/PR

Equipe (Squad): Giovanni Henrique Gazin Pasa — RA 25355659-2 | Gabriel Aguitoni Lázaro De Azevedo — RA 24503163-2 | Gustavo Henrique Portel de Almeida — RA 25364235-2

ODS contemplada: ODS 2 — Fome Zero e Agricultura Sustentável (com aderência à meta 12.3 da ODS 12)

1. Descoberta e Concepção: o problema e a ODS

O ponto de partida deste projeto foi uma cena bastante comum no comércio de bairro. Todo fim de expediente, o gerente de um mini mercado separa caixas de frutas amassadas, pães do dia, hortifrúti fora do padrão estético e produtos a um ou dois dias do vencimento. Nada disso está estragado, mas também não pode mais ser vendido. O destino quase sempre é o mesmo: o lixo. A poucos quarteirões dali, uma casa de apoio social monta o cardápio da semana contando moeda e improvisando o que vai faltar.

O que existe hoje para ligar esses dois lados é a informalidade. A combinação acontece por grupo de WhatsApp ou por telefone, sem registro do que foi doado, sem previsão de quem vai buscar e sem qualquer controle de prazo. O resultado é previsível: doações que ninguém retira a tempo e acabam estragando, instituições que aparecem para buscar algo que outra já levou, e nenhum histórico que permita ao estabelecimento provar o quanto deixou de desperdiçar. A janela de oportunidade é curta, quase sempre menor que 48 horas, e é exatamente essa janela que a informalidade desperdiça.

Segundo a FAO, cerca de um terço de todo o alimento produzido no mundo é perdido ou desperdiçado ao longo da cadeia, enquanto milhões de pessoas seguem em insegurança alimentar. O Sobra Zero ataca o último elo dessa cadeia, o varejo de bairro, aproximando quem tem excedente de quem tem demanda. A conexão com a ODS 2 (Fome Zero e Agricultura Sustentável) é direta: cada quilo redirecionado é comida que chega à mesa em vez de ir para o aterro. Como consequência, o projeto também conversa com a meta 12.3 da ODS 12, que trata da redução do desperdício de alimentos no varejo.

1.1 Resumo do Discovery (Matriz CSD)

Antes de modelar o sistema, a equipe organizou o entendimento do problema em uma Matriz CSD. As conclusões que mais impactaram a arquitetura foram:

- **Certeza:** o valor de uma doação cai com o tempo. Uma bandeja de tomates disponível hoje vale muito; a mesma bandeja em três dias vale zero. Qualquer listagem por ordem de cadastro seria inútil.

- **Suposição validada:** o estabelecimento não vai preencher formulário longo no meio da correria do fechamento. O cadastro de doação precisa ser curto e direto, com poucos campos obrigatórios.
- **Dúvida resolvida:** duas instituições poderiam reivindicar a mesma doação? A resposta define a regra mais crítica do sistema — uma doação reservada precisa ser bloqueada para as demais, sob pena de gerar viagem perdida e desconfiança na plataforma.

Dessas conclusões nasceu a premissa técnica central do projeto: o sistema é, no fundo, uma fila de prioridade governada pela validade, com controle transacional da reserva e rastreabilidade completa do ciclo de vida de cada doação (DISPONÍVEL, AGENDADA, CONCLUÍDA ou EXPIRADA). É essa premissa que justifica, mais adiante, a escolha de um banco relacional e de uma hierarquia polimórfica de doações.

2. Lista de Requisitos Funcionais (escopo)

O escopo abaixo delimita o que o sistema fará no 2º bimestre. Foram priorizados os requisitos que sustentam o fluxo principal, da publicação do excedente até a confirmação da retirada.

- **RF01** — O sistema deve permitir o cadastro, a consulta, a edição e a exclusão de usuários em três perfis distintos (Estabelecimento, Instituição e Administrador), contendo nome, e-mail, senha, CNPJ e endereço, com autenticação por e-mail e senha no acesso.
- **RF02** — O sistema deve permitir que o estabelecimento registre uma doação informando o tipo (perecível ou não perecível) e a lista de itens que a compõem, com descrição, quantidade e unidade de medida. Para doações perecíveis, a data de validade e a necessidade de refrigeração passam a ser obrigatórias.
- **RF03** — O sistema deve calcular automaticamente a prioridade de cada doação disponível, considerando a proximidade da data de validade e a exigência de refrigeração, e exibir o painel de doações ordenado da mais urgente para a menos urgente.
- **RF04** — O sistema deve permitir que uma instituição reserve uma doação disponível, informando a data prevista de retirada, alterando o status para AGENDADA e impedindo que outra instituição reserve a mesma doação.
- **RF05** — O sistema deve permitir a confirmação da retirada pelo estabelecimento, registrando data e hora do evento e movendo a doação para o status CONCLUÍDA, o que consolida o peso total resgatado.
- **RF06** — O sistema deve identificar as doações perecíveis não retiradas até a data de validade, marcá-las como EXPIRADA e removê-las do painel de disponíveis, preservando o registro para fins de análise.
- **RF07** — O sistema deve gerar relatórios consolidados por período, apresentando o total de quilos resgatados, a quantidade de doações por status e o desempenho individual de cada estabelecimento e instituição, com opção de exportação em arquivo CSV.

2.1 Requisitos Não Funcionais

- **RNF01** — A aplicação será executada via terminal (console), sem dependência de navegador ou servidor de aplicação, priorizando a portabilidade na máquina do avaliador.
- **RNF02** — As senhas serão armazenadas exclusivamente na forma de hash, nunca em texto puro.
- **RNF03** — Todos os dados serão persistidos em banco relacional; nenhuma informação de negócio permanecerá apenas em memória durante a execução.
- **RNF04** — O código será organizado em camadas independentes (model, dao, service e view), de modo que a substituição futura da interface de console por uma interface web não exija reescrita das regras de negócio.

3. Planejamento Ágil (Cronograma / Backlog)

O trabalho do 2º bimestre foi fatiado verticalmente: cada sprint entrega uma funcionalidade completa, da tela de console até a gravação no banco, em vez de entregar camadas soltas que só fariam sentido no final. A sequência segue a dependência técnica natural — não existe doação sem usuário autenticado, nem reserva sem doação cadastrada.

Sprint / Período	Épico	User Story	Responsável
Sprint 1 14/09 a 27/09	Fundação e Controle de Acesso	COMO UM administrador EU QUERO cadastrar e autenticar estabelecimentos, instituições e administradores PARA QUE cada perfil acesse apenas as funções que lhe competem.	Giovanni
Sprint 2 28/09 a 11/10	Núcleo de Doações (CRUD)	COMO UM estabelecimento EU QUERO registrar, consultar, editar e excluir minhas doações com seus respectivos itens PARA QUE eu consiga anunciar rapidamente o excedente antes que ele estrague.	Gustavo
Sprint 3 12/10 a 25/10	Priorização e Agendamento	COMO UMA instituição receptora EU QUERO visualizar as doações disponíveis ordenadas por urgência e reservar uma coleta PARA QUE eu retire primeiro aquilo que vence antes.	Gabriel
Sprint 4 26/10 a 08/11	Relatórios, Expiração e Fechamento	COMO UM administrador EU QUERO acompanhar o volume resgatado por período e ver as doações expiradas automaticamente PARA QUE eu consiga medir o impacto real da plataforma.	Giovanni, Gabriel e Gustavo

Para garantir a rastreabilidade da participação individual exigida pela disciplina, cada integrante fará os commits do próprio módulo em uma branch nomeada (por exemplo, feature/doacoes), com integração à branch principal via Pull Request revisado por outro membro da equipe. As revisões de fim de sprint ficam registradas no README do repositório.

4. Justificativa Técnica e Arquitetural

4.1 Linguagem: Java 21 (LTS)

A escolha do Java não foi por familiaridade, e sim pelo encaixe com o domínio. As regras do Sobra Zero são rígidas e precisam falhar cedo: uma doação perecível sem data de validade é um erro de modelagem, não uma variação aceitável. A tipagem estática do Java transforma esse tipo de inconsistência em erro de compilação, antes que ele vire um registro sujo no banco. Além disso, a linguagem oferece nativamente todos os recursos que o projeto precisa exercitar — classes abstratas, interfaces, sobrescrita com `@Override` e enums — sem depender de convenções ou de bibliotecas externas. Por ser uma versão LTS e multiplataforma, o Java 21 também reduz o risco de o projeto não compilar na máquina do avaliador.

4.2 Acesso a dados: JDBC com padrão DAO, sem ORM

A equipe optou conscientemente por não utilizar um framework ORM nesta etapa. Como o foco da Engenharia de Software neste semestre está no rigor arquitetural, escrever o acesso a dados com JDBC e o padrão DAO obriga a lidar explicitamente com conexão, `PreparedStatement`, `ResultSet` e tratamento de transação — justamente o que um ORM esconderia. O Maven entra apenas como gerenciador de dependências mínimas (driver do PostgreSQL e biblioteca de hash de senha), mantendo o build reproduzível com um único comando.

4.3 Banco de dados: PostgreSQL

O domínio do problema é essencialmente relacional: um estabelecimento tem várias doações, uma doação tem vários itens e gera no máximo uma coleta. Essas cardinalidades precisam ser garantidas pelo próprio banco, e não apenas pela aplicação. O PostgreSQL entrega chaves estrangeiras com `ON DELETE CASCADE` (excluir uma doação remove seus itens sem deixar lixo), restrições `CHECK` para validar os status permitidos e uma restrição `UNIQUE` sobre `doacao_id` na tabela de coletas, que é a rede de segurança contra o problema de duas instituições reservarem a mesma doação.

Uma alternativa NoSQL orientada a documentos foi avaliada e descartada. Ela facilitaria o armazenamento dos itens dentro do próprio documento da doação, mas custaria caro justamente onde o projeto é mais sensível: consistência transacional na reserva e integridade referencial entre entidades. Somado a isso, o PostgreSQL é gratuito, multiplataforma e amplamente documentado, o que simplifica a reprodução do ambiente pelo professor a partir do script disponível em `/database`.

4.4 Arquitetura e aplicação dos pilares da POO

A aplicação segue uma separação em quatro camadas. A camada view concentra os menus de console e a leitura de entradas; a camada service guarda as regras de negócio (cálculo de prioridade, validação da reserva, expiração); a camada dao isola o SQL; e a camada model contém as entidades de domínio. A interface de console é deliberadamente a parte mais descartável do sistema — trocá-la não deve exigir mudança em nenhuma outra camada.

- **Abstração e Herança:** a classe abstrata `Usuario` centraliza identidade, e-mail e autenticação, e é estendida por `Estabelecimento`, `Instituicao` e `Administrador`. O mesmo raciocínio se repete em `Doacao`, abstrata, estendida por `DoacaoPercivel` e `DoacaoNaoPercivel`.
- **Encapsulamento:** todos os atributos são `private`, com acesso controlado por métodos públicos. O status da doação, em especial, só muda através de métodos de transição da camada de serviço, o que impede que um código externo coloque a entidade em um estado inválido.
- **Polimorfismo:** os métodos `calcularPrioridade()` e `descreverManuseio()` são declarados em `Doacao` e sobrescritos com `@Override` nas subclasses. A doação `perceivel` calcula sua urgência a partir das horas restantes até o vencimento; a não `perceivel` devolve prioridade baixa e constante. A camada de serviço apenas percorre uma `List<Doacao>` e ordena, sem precisar saber com qual tipo está lidando — acrescentar um novo tipo de doação no futuro não exige alterar o código já existente.
- **Composição 1:N:** um `Estabelecimento` mantém a lista das suas doações, e cada `Doacao` é composta por uma lista de `ItemDoacao`. Os itens não têm existência própria fora da doação, o que se reflete tanto no diagrama (losango preenchido) quanto no banco (exclusão em cascata).

5. Estruturação do GitHub e Diagramas

5.1 Repositório

Repositório público: github.com/giovannihgp/sobra-zero-2026

A estrutura de pastas já se encontra criada no repositório, preparada para receber o código do 2º bimestre:

/src — código-fonte Java, subdividido em model, dao, service e view.

/docs — este documento, os arquivos-fonte dos diagramas e as atas de revisão de sprint.

/database — script schema.sql com a criação das tabelas e um seed.sql com dados de exemplo para teste.

README.md — instruções de execução: versão do JDK, criação do banco, execução do script e configuração da conexão.

5.2 Diagrama de Classes (UML)

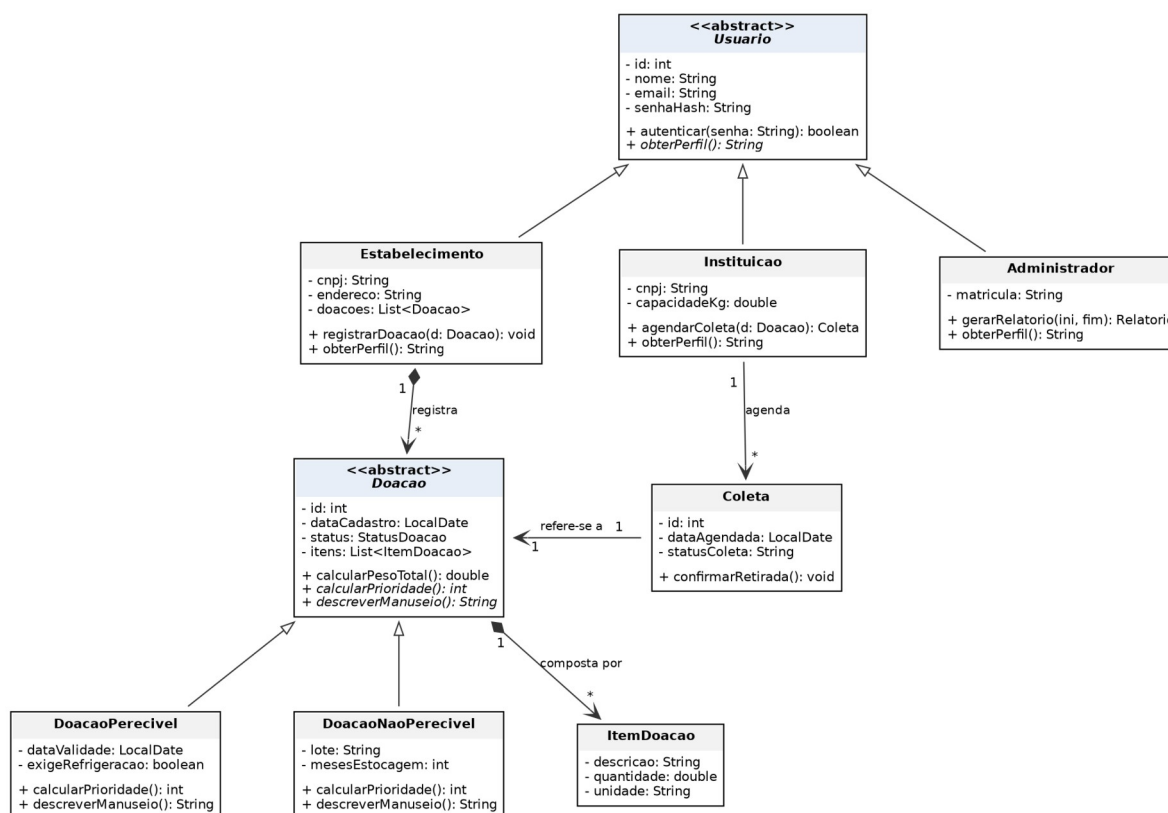


Figura 1 — Diagrama de classes do Sobra Zero.

- **Herança:** a superclasse abstrata **Usuario** reúne os atributos comuns (id, nome, email, senhaHash) e o comportamento de autenticação. Dela derivam **Estabelecimento**, **Instituicao** e **Administrador**, cada uma com seus dados e responsabilidades específicos. A mesma estratégia se aplica à hierarquia de **Doacao**.

- **Composição 1:N:** Estabelecimento contém uma lista de Doacao (um mercado publica várias doações ao longo do tempo) e cada Doacao é composta por vários objetos ItemDoacao, que só existem enquanto a doação existir.
- **Polimorfismo:** a assinatura calcularPrioridade() é declarada na classe abstrata Doacao e sobrescrita com @Override por DoacaoPerecivel e DoacaoNaoPerecivel, que aplicam lógicas de urgência completamente distintas para a mesma chamada de método.

5.3 Diagrama de Banco de Dados (DER)

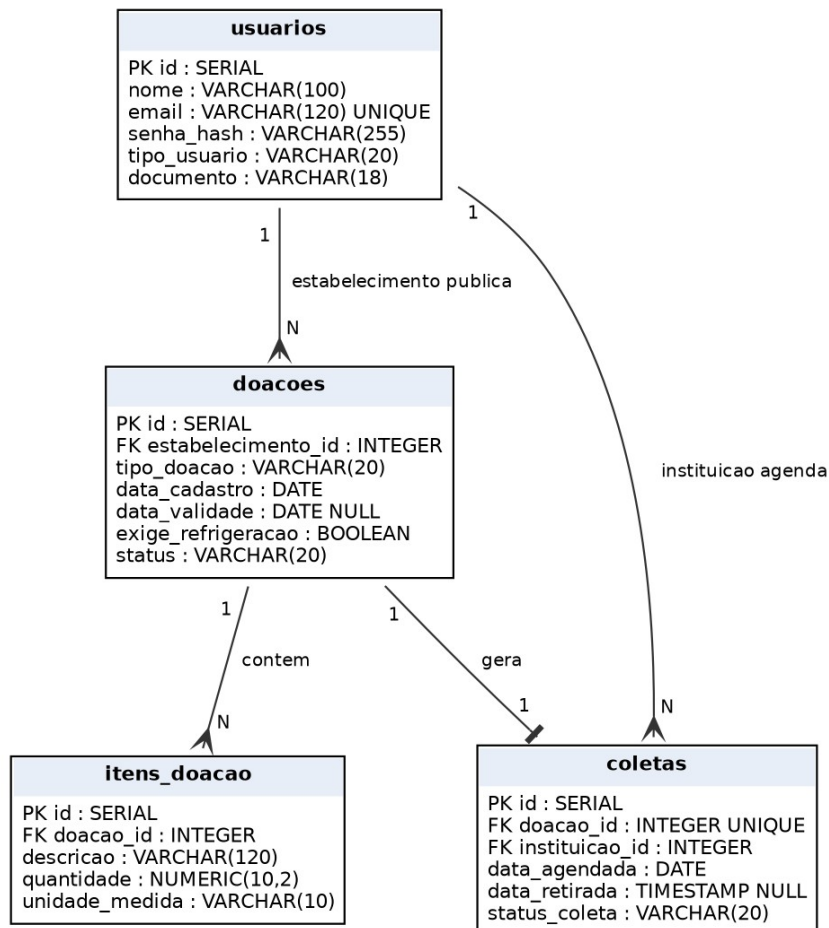


Figura 2 — Modelo relacional do Sobre Zero.

- **Tabela usuarios:** centraliza credenciais e dados cadastrais dos três perfis, utilizando a coluna discriminadora tipo_usuario para diferenciá-los (estratégia de tabela única para a hierarquia de herança).
- **Tabela doacoes:** registra cada anúncio de excedente, com a chave estrangeira estabelecimento_id, a coluna tipo_doacao que espelha a hierarquia polimórfica do modelo e a coluna status, que controla o ciclo de vida do registro.
- **Tabela itens_doacao:** detalha o conteúdo de cada doação. A chave estrangeira doacao_id é configurada com ON DELETE CASCADE, traduzindo no banco a composição forte representada no diagrama de classes.

- **Tabela coletas:** materializa o encontro entre a doação e a instituição que a reservou. A restrição UNIQUE sobre doacao_id garante, no nível do banco, que uma mesma doação nunca seja reservada duas vezes.